Lina Harcharras

Milestone 1:

1.1 System architecture:

The Distributed Log Analysis Platform (DLAP) follows a client–server architecture.

- The client provides a simple UI for sending analysis requests (count, status) and displaying results.
- The server receives requests, processes the specified CLF log file, performs the analysis remotely, and returns results.
- Communication occurs through TCP sockets using custom text-based headers.

Flow:
Client connects → Client sends request → Server analyzes log → server replies → Client displays output.

1.2 Contract / Protocol:

Once the connection is established, the client sends a request to the server to inform it about which analysis it wants to perform using a header.

If the client wants to get the total number of requests in a log file, then the header will be as follows:

COUNT[one space][file name][Line Feed]

Upon receiving the header, the server searches for the specified log file.
If the file is not found, the server shall reply with the following header:

NOT[one space]FOUND[Line Feed]

Else, the server replies with a header followed by the bytes of the analysis result:

OK[one space][number of requests][Line Feed]

If the client wants to get the distribution of HTTP status codes, then the header will be as follows:

STATUS[one space][file name][Line Feed]

Upon receiving this header, the server searches for the specified log file.

If the file is not found, the server shall reply with:

NOT[one space]FOUND[Line Feed]

Else, the server shall reply with a header followed by the bytes of the analysis result:

OK[one space][status code][one space][occurrence count][Line Feed]

1.3 Implementation (Python):

Please refer to P1server.py and P2client.py in the file P1 in the zip file.

Milestone2:

2.1 Technology Selection and Justification:

Selected Technology: SOAP with XML

Justification:

Language Independence: SOAP works seamlessly between Java and Python through standardized WSDL contracts

Formalized Contract: WSDL provides explicit interface definitions that both languages understand

Built-in Type Safety: XML Schema validates data types automatically

Higher Abstraction: Client code appears local - hides network complexity

Enterprise Standard: Well-supported in both Java (JAX-WS) and Python (Zeep/suds)

Suitable for Assessments: Clear patterns that are reproducible in exam settings

2.2 WSDL contract (design)

Please refer to wsdl.txt in the zip file.

2.3 Implementation

Please refer to the remaining code in the zip file:
LogAnalysisClient.java and soap_server.py